

PROJECT DOCUMENTATION

Project Name:

Secure CI/CD Pipeline with Zero-Downtime Deployment on AWS

Developed by:

Mukhesh DN

Muppidi Sai Adhitya

S. No.	TOPIC	Page No
1	Purpose Of This Documentation	1
2	High Level Design	2
3	Docker Setup	3-6
4	Jenkins Setup	7-24
5	Kubernetes Setup	25-28
6	ArgoCD Setup	29-32
7	Prometheus And Grafana Setup	33-36

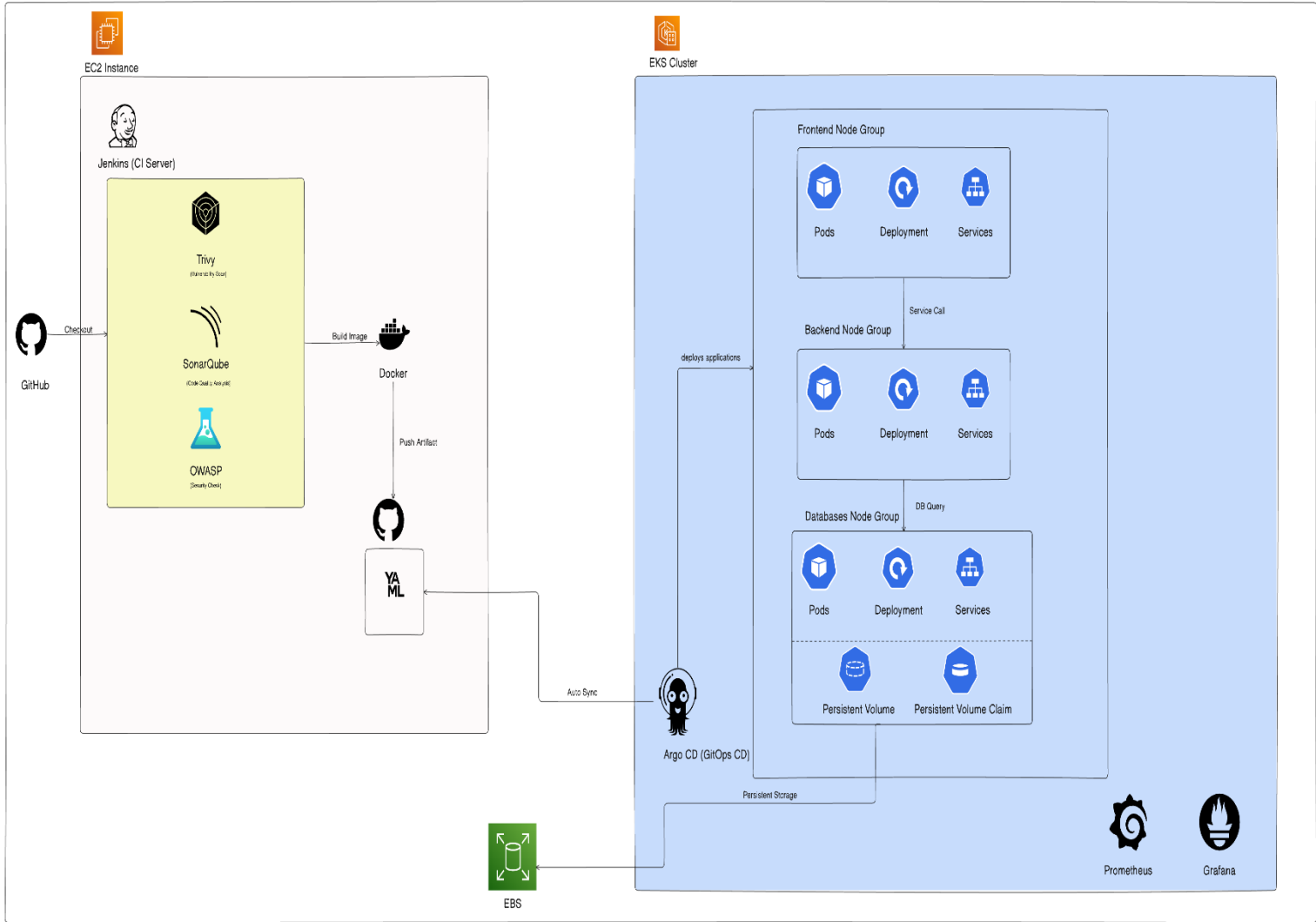
Purpose Of This Documentation

The purpose of this document is to serve as a detailed reference for the design, implementation, and execution of the DevSecOps pipeline. It captures:

- The high-level architecture and workflow of the CI/CD process
- The tools and technologies used at each stage
- Step-by-step procedures followed during implementation
- Commands executed and configurations applied
- Security controls and quality checks integrated into the pipeline
- Deployment strategy and GitOps workflow

This documentation is **intended** for developers, DevOps engineers, security engineers, and stakeholders who want to understand, replicate, maintain, or enhance the implemented DevSecOps solution. It also serves as a knowledge base for onboarding new team members and for future audits or enhancements of the pipeline.

High level design



Docker Setup

PreRequisites

Before proceeding with the Docker setup and containerization of the application, ensure that the following prerequisites are in place:

1. **Docker Hub Account**
An active Docker Hub account is required to store and manage container images for both frontend and backend services.
2. **Separate Docker Hub Repositories (Frontend & Backend)**
Dedicated Docker Hub repositories for the frontend and backend services to enable independent image versioning and deployment.
3. **Docker Desktop**
Docker Desktop must be installed and running on the local development machine or build server for building and testing Docker images.
4. **Docker CLI**
Docker Command Line Interface (CLI) installed and accessible from the terminal to execute build, tag, login, and push commands.

Clone the Source Code from GitHub

Link: <https://github.com/MukheshDN4352/DevSecOps-Project.git>

Dockerfile Design and Configuration Overview

1. We used an open-source **three-tier application** named **Social Echo** as the base application for this project.
2. A **multi-stage Docker build** approach was implemented to reduce the final **Docker image size** and improve build efficiency.
3. For the frontend service, **Nginx** was used as the **web server**.
 - a. The lightweight **Alpine-based Nginx image (nginx:alpine)** was selected as the runtime base image to minimize resource usage.
 - b. **Nginx** was configured to serve the **frontend application**.
4. All required **Dockerfiles** for the frontend and backend services, along with the **Nginx configuration file (nginx.conf)**, are provided below for reference.

FrontEnd Dockerfile

FROM node:18-alpine AS build

Set working directory

WORKDIR /app

Copy dependency files

COPY package*.json ./

Install dependencies (clean, reproducible install)

RUN npm ci

Add Babel plugin (for CRA compatibility with private props)

RUN npm install --save-dev @babel/plugin-proposal-private-property-in-object

Copy source code

COPY . .

Build production assets

RUN npm run build

#-----

FROM nginx:alpine

Copy built static files from build stage

COPY --from=build /app/build /usr/share/nginx/html

Remove default nginx config and replace with a custom one

COPY nginx.conf /etc/nginx/conf.d/default.conf

Expose port 80

EXPOSE 80

Start Nginx in the foreground

CMD ["nginx", "-g", "daemon off;"]

Nginx config file

```
server {  
    listen 80;  
    server_name _;  
  
    root /usr/share/nginx/html;  
    index index.html;  
  
    # Handle SPA routes (React Router)  
    location / {  
        try_files $uri /index.html;  
    }  
  
    # Proxy API calls to backend  
    location /api/ {  
        proxy_pass http://backend:4000;  
        proxy_http_version 1.1;  
        proxy_set_header Upgrade $http_upgrade;  
        proxy_set_header Connection 'upgrade';  
        proxy_set_header Host $host;  
        proxy_cache_bypass $http_upgrade;  
    }  
  
    # Error handling  
    error_page 500 502 503 504 /50x.html;  
    location = /50x.html {  
        root /usr/share/nginx/html;  
    }  
}
```

Backend Dockerfile

```
# ---- Stage 1: Build dependencies ----  
FROM node:18-alpine AS build  
# Set working directory  
WORKDIR /usr/src/app  
# Copy dependency files first (for better caching)  
COPY package*.json ./  
# Install all dependencies (including dev)  
RUN npm install  
# Copy the application code  
COPY . .  
  
# ---- Stage 2: Production runtime ----  
FROM node:18-alpine  
# Set working directory  
WORKDIR /usr/src/app  
# Copy only production dependencies from the build stage  
COPY --from=build /usr/src/app/node_modules ./node_modules  
# Copy source code from build stage  
COPY --from=build /usr/src/app ./  
# Expose port (backend runs on 4000)  
EXPOSE 4000  
# Set environment variable for production  
ENV NODE_ENV=production  
# Start the app using npm production script  
CMD ["npm", "run", "production"]
```

Jenkins Setup

PreRequisites

- **AWS Account**
An active AWS account to provision and manage the Jenkins server (EC2 instance) and related resources.
- **EC2 Instance**
An Amazon EC2 instance (Linux-based) to host the Jenkins server.
- **IAM Role or User Credentials**
AWS IAM role or user with required permissions to:
Access EC2

Jenkins Setup and Pipeline Overview

In this project, **Jenkins** is hosted on an **AWS EC2 instance** and accessed through its **public IP address**. Jenkins is used as the core **CI engine** to automate the build, security scanning, and container image creation processes.

A **canary deployment model** is implemented to enable safe and incremental releases. New application versions are first deployed to a **canary environment** for validation before being promoted to the **stable environment**.

The Jenkins pipeline consists of the following stages:

- **Clone Code from GitHub**
- **Promote Existing Canary → Stable**
- **SonarQube Quality Analysis**
- **OWASP Dependency Check**
- **Sonar Quality Gate**
- **Trivy File System Scan**
- **Build & Push Docker Images**
- **Update Canary with New Images**
- **Commit & Push to GitHub (for ArgoCD Sync)**

Post-build actions include **email notifications** for both **successful** and **failed** pipeline executions.

All required **Jenkins credential configurations**, **plugins installation**, **pipeline configurations**, **EC2 instance setup commands**, and **GitHub webhook configuration for automatic build triggers** are provided in the sections below.

Creating EC2 instance in AWS

EC2 > Instances > Launch an instance

Launch an instance Info

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags Info

Name

[Add additional tags](#)

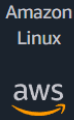
▼ Application and OS Images (Amazon Machine Image) Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Q Search our full catalog including 1000s of application and OS images

Recents

Quick Start



Amazon Linux



macOS



Ubuntu



Windows



Red Hat



SUSE Linux



Debian



[Browse more AMIs](#)

Including AMIs from AWS, Marketplace and the Community

Select **ubuntu** AMI

▼ Instance type Info | [Get advice](#)

Instance type

t2.xlarge

Family: t2 4 vCPU 16 GiB Memory Current generation: true

On-Demand SUSE base pricing: 0.2984 USD per Hour On-Demand Linux base pricing: 0.1984 USD per Hour

On-Demand RHEL base pricing: 0.256 USD per Hour

On-Demand Ubuntu Pro base pricing: 0.2054 USD per Hour

On-Demand Windows base pricing: 0.2394 USD per Hour

☐ All generations

[Compare instance types](#)

Additional costs apply for AMIs with pre-installed software

▼ Key pair (login) Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - *required*

[Create new key pair](#)

Select **T2.xlarge** instance type

▼ Network settings Info

Edit

Network Info

vpc-0deab8a175d25a4db

Subnet Info

No preference (Default subnet in any availability zone)

Auto-assign public IP Info

Enable

Firewall (security groups) Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group

☐ Select existing security group

We'll create a new security group called 'launch-wizard-6' with the following rules:

☒ Allow SSH traffic from

Helps you connect to your instance

Anywhere

0.0.0.0/0

☒ Allow HTTPS traffic from the internet

To set up an endpoint, for example when creating a web server

☒ Allow HTTP traffic from the internet

To set up an endpoint, for example when creating a web server

▼ Configure storage Info

Advanced

1x 20 GiB gp3

Root volume, 3000 IOPS, Not encrypted

Add new volume

The selected AMI contains instance store volumes, however the instance does not allow any instance store volumes. None of the instance store volumes from the AMI will be accessible from the instance

Click refresh to view backup information

The tags that you assign determine whether the instance will be backed up by any Data Lifecycle Manager policies.

0 x File systems

Edit

Take 20 GIB volume

▼ Summary

Number of instances Info

1

Software Image (AMI)

Canonical, Ubuntu, 24.04, amd64...read more

ami-02b8269d5e85954ef

Virtual server type (instance type)

t2.xlarge

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 20 GiB

Cancel

Launch instance

Preview code

Click on launch instance

Connect Info
Connect to an instance using the browser-based client.

EC2 Instance Connect Session Manager **SSH client** EC2 serial console

Instance ID
i-06250445eb62f9bd1 (Jenkins-server)

1. Open an SSH client.
2. Locate your private key file. The key used to launch this instance is jenkins-demo.pem
3. Run this command, if necessary, to ensure your key is not publicly viewable.
`chmod 400 "jenkins-demo.pem"`
4. Connect to your instance using its Public DNS:
`ec2-13-233-108-63.ap-south-1.compute.amazonaws.com`

Example:
`ssh -i "jenkins-demo.pem" ubuntu@ec2-13-233-108-63.ap-south-1.compute.amazonaws.com`

Note: In most cases, the guessed username is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

Copy this command to connect to the instance from your cmd

```
Command Prompt
C:\Users\mukes\Downloads>ssh -i "jenkins-demo.pem" ubuntu@ec2-13-233-108-63.ap-south-1.compute.amazonaws.com
```

```
ubuntu@ip-172-31-41-224: ~
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1015-aws x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Tue Jan 20 05:01:50 UTC 2026

System load:  0.06          Processes:      110
Usage of /:   9.4% of 18.33GB Users logged in: 0
Memory usage: 43%          IPv4 address for enx0: 172.31.41.224
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Tue Jan 20 05:01:51 2026 from 115.99.242.65
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-41-224:~$
```

The next set of commands needs to be executed on the Ubuntu EC2 instance.

Jenkins-server Configuration Commands

`sudo apt-get update`

docker installation

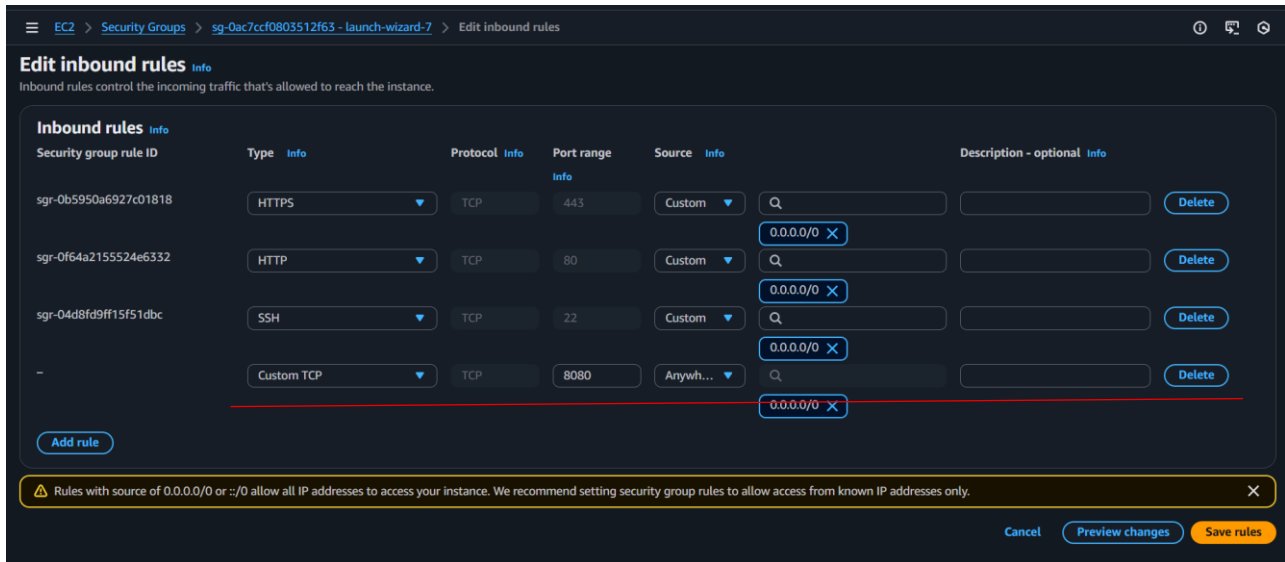
1. `sudo apt-get install docker.io -y`
2. `sudo apt-get install docker-compose -y`
3. `sudo usermod -aG docker ubuntu`
4. `sudo reboot`
5. `docker ps`

jenkins installation

1. `sudo apt update`
2. `sudo apt install fontconfig openjdk-21-jre`
3. `java -version`
4. `sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null`
5. `sudo apt update`
6. `sudo apt install jenkins`
7. `sudo systemctl status jenkins`
8. `wq`

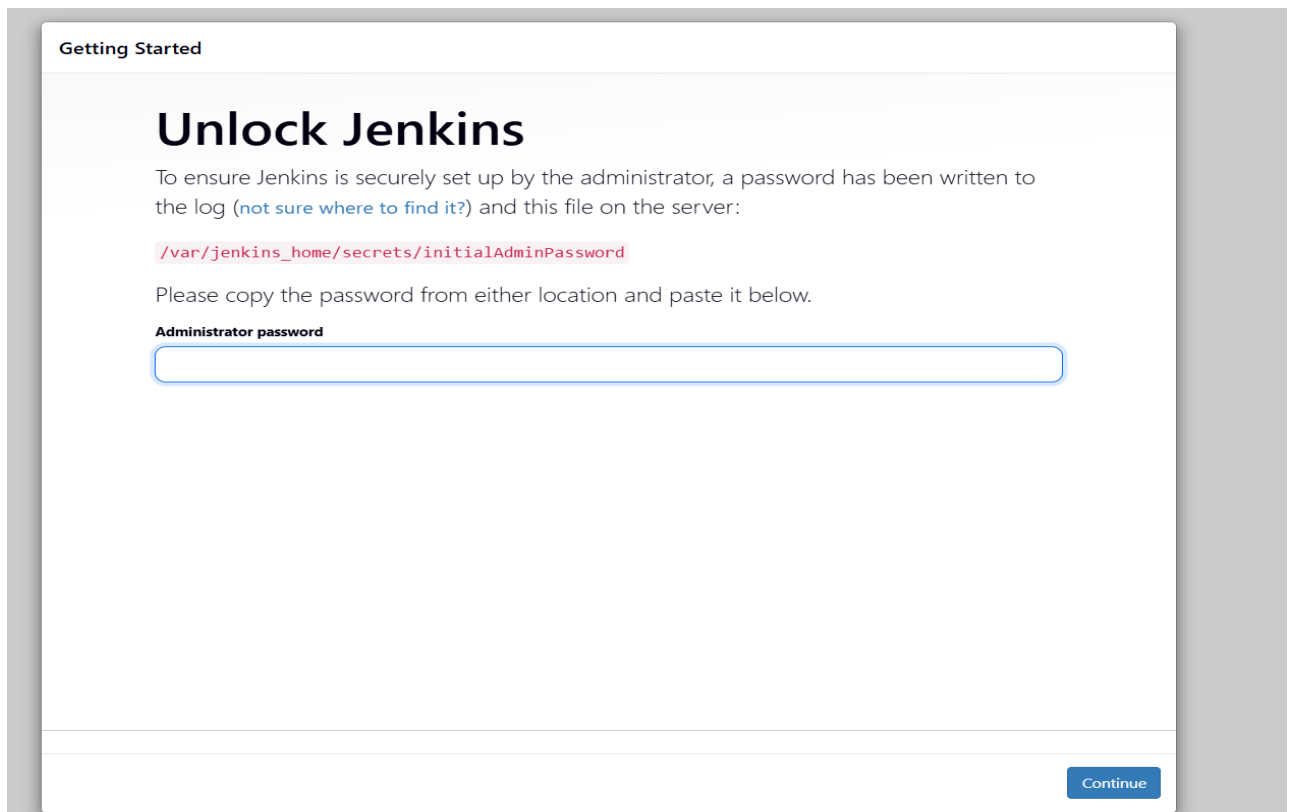
Allow inbound traffic to the instance

aws → ec2 instance → security groups → edit inbound rules → allow port 8080



Access the jenkins application through

<http://<public IP of the instance>:8080>



```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

plugins to install

sonarQube Scanner

SonarQuality gates

owasp dependency check

docker

stage view

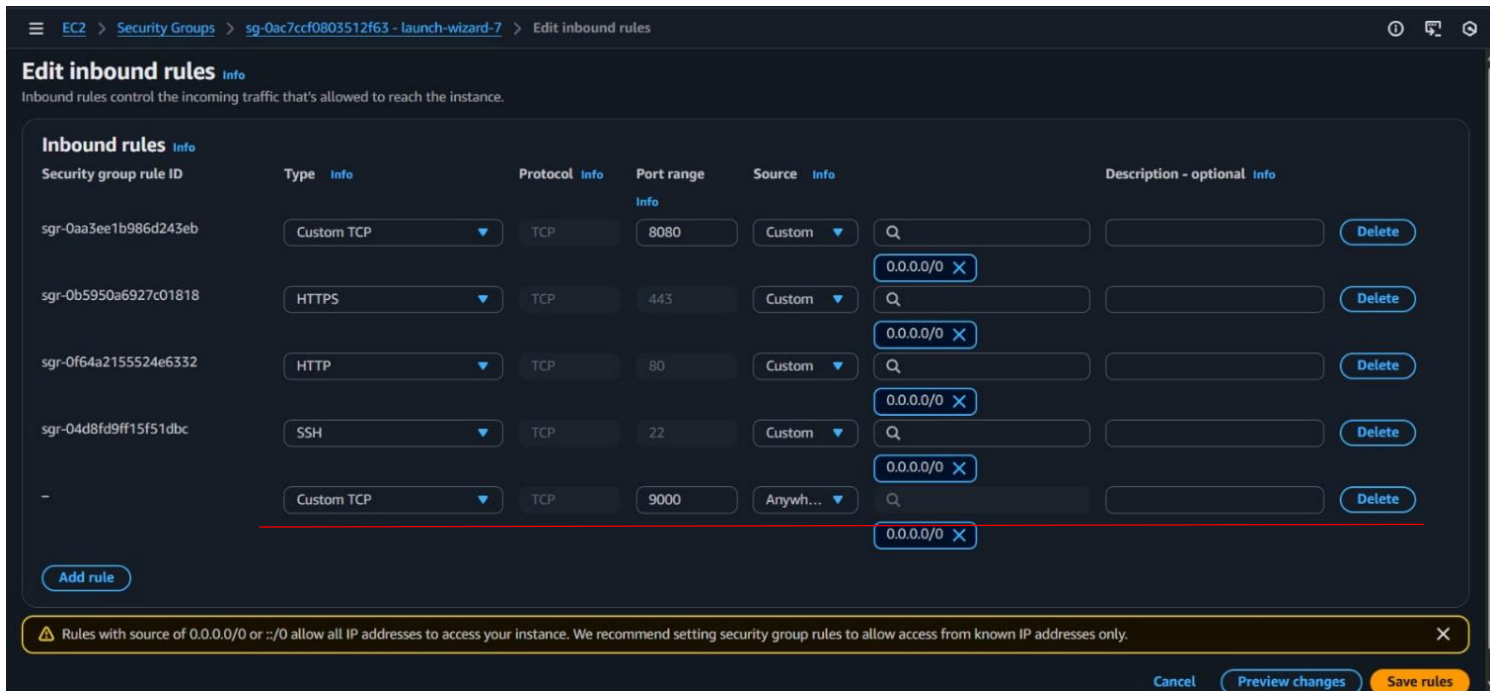
HTML publisher (for Trivy)

Updates	Install	Name ↓	Released	Health
Available plugins	✓	SonarQube Scanner 2.18.2 External Site/Tool Integrations Build Reports This plugin allows an easy integration of SonarQube , the open source platform for Continuous Inspection of code quality.	1 mo 3 days ago	84
Installed plugins	✓	Sonar Quality Gates 364.v67a_f255f340f Library plugins (for use by other plugins) analysis Other Post-Build Actions Fails the build whenever the Quality Gates criteria in the Sonar 5.6+ analysis aren't met (the project Quality Gates status is different than "Passed")	26 days ago	100
Advanced settings	✓	OWASP Dependency-Check 5.6.2 Security DevOps Build Tools Build Reports This plug-in can independently execute a Dependency-Check analysis and visualize results. Dependency-Check is a utility that identifies project dependencies and checks if there are any known, publicly disclosed, vulnerabilities.	1 mo 29 days ago	100
Download progress	✓	Docker 1308.vff6e33248305 Cloud Providers Cluster Management docker This plugin integrates Jenkins with Docker	1 mo 23 days ago	100
	✓	Pipeline: Stage View 2.39 User Interface Pipeline Stage View Plugin.	1 day 8 hr ago	100
	✓	HTML Publisher 427 Build Reports This plugin publishes HTML reports.	6 mo 14 days ago	97

Sonarqube Setup

```
docker run -itd --name sonarqube-server -p 9000:9000 sonarqube:lts-community
```

aws ---> ec2 instance --> security groups ---> edit inbound rules ---> allow port 9000



Trivy installation

- `sudo apt-get install wget apt-transport-https gnupg lsb-release`
- `wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | \`
`gpg --dearmor | sudo tee /usr/share/keyrings/trivy.gpg`
- `echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] https://aquasecurity.github.io/trivy-`
`repo/deb \`
`$(lsb_release -sc) main" | sudo tee /etc/apt/sources.list.d/trivy.list`
- `sudo apt-get update`
- `sudo apt-get install trivy`

Sonarqube and jenkins integeation

1)sonarqube → administrator →configuration → webhook → create →

name: jenkins

url :http://35.154.237.213:8080/sonarqube-webhook/

An HTTP

Create Webhook

All fields marked with * are required

Name *

URL *

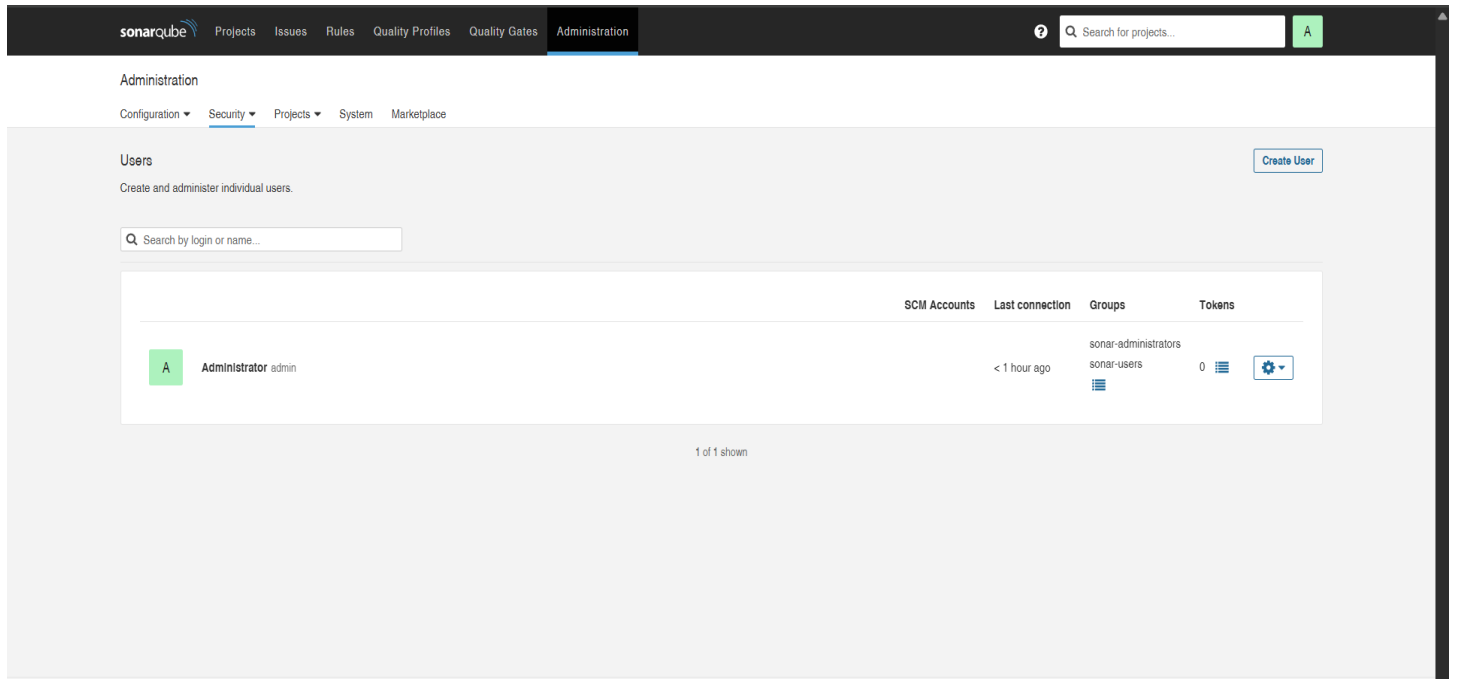
Server endpoint that will receive the webhook payload, for example:
"http://my_server/foo". If HTTP Basic authentication is used, HTTPS is recommended to avoid man in the middle attacks. Example:
"https://myLogin:myPassword@my_server/foo"

Secret

If provided, secret will be used as the key to generate the HMAC hex (lowercase) digest value in the 'X-Sonar-Webhook-HMAC-SHA256' header.

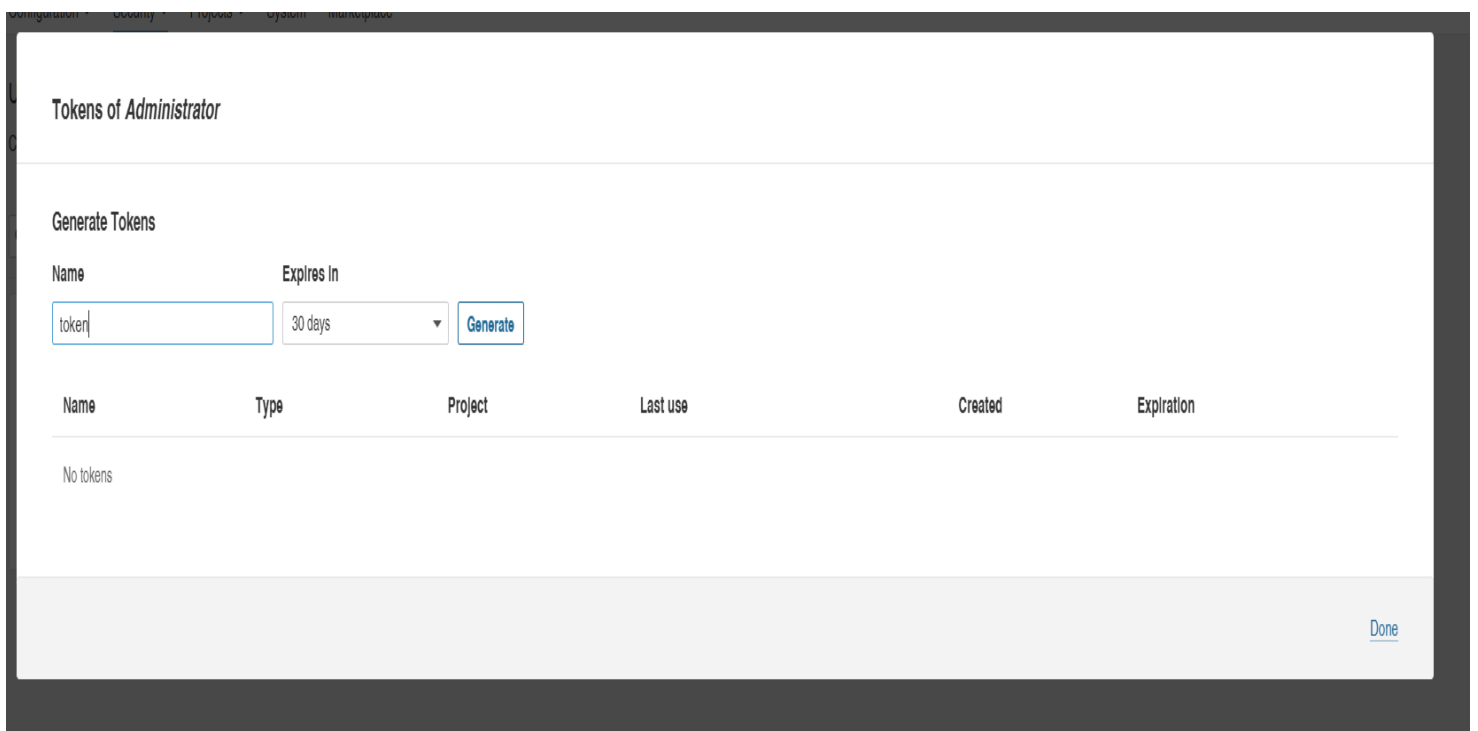
[Create](#) [Cancel](#)

2)sonarqube --> administrator --> security --> user --> generate token
[ex:squ_1d7e9d17fa42aebfaaab8846aefcbbedb8c33b60]



The screenshot shows the SonarQube Administration interface. The top navigation bar includes links for Projects, Issues, Rules, Quality Profiles, Quality Gates, and Administration. The Administration section is active, and the left sidebar shows Configuration, Security, Projects, System, and Marketplace. The main content area is titled 'Users' and includes a 'Create User' button. Below this is a search bar labeled 'Search by login or name...'. A table lists the users, with one user 'Administrator' (admin) shown. The table has columns for SCM Accounts, Last connection, Groups, and Tokens. The 'Administrator' user has a last connection of '< 1 hour ago' and is associated with the 'sonar-administrators' and 'sonar-users' groups. The 'Tokens' column shows '0' tokens. A '1 of 1 shown' indicator is at the bottom of the table.

	SCM Accounts	Last connection	Groups	Tokens
 Administrator admin		< 1 hour ago	sonar-administrators sonar-users	0



The screenshot shows the 'Tokens of Administrator' page. It has a title 'Tokens of Administrator' and a section 'Generate Tokens'. Below this is a form with a 'Name' field containing 'token', an 'Expires In' dropdown set to '30 days', and a 'Generate' button. Below the form is a table with columns: Name, Type, Project, Last use, Created, and Expiration. The table currently shows 'No tokens'. A 'Done' button is at the bottom right.

Name	Type	Project	Last use	Created	Expiration
No tokens					

3)jenkins → credentials →global →add credentials →

[secret text] id : generated token,

description :Sonar

New credentials

Kind

Secret text

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Secret

....

ID ?

Sonar

Description ?

Sonar

Create

4)jenkins --> system --> SonarQube installations -->

NAME: sonar ,

SERVER URL:http://35.154.237.213:9000 ,

SERVER_AUTH_Token :sonar

SonarQube servers

If checked, job administrators will be able to inject a SonarQube server configuration as environment variables in the build.

☐ Environment variables

SonarQube installations

List of SonarQube installations

Name

Sonar

Server URL

Default is http://localhost:9000

http://localhost:9000

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

Sonar

+ Add

Advanced

+ Add SonarQube

5)jenkins → tools →sonarqube scanner

The screenshot shows the Jenkins 'Tools' configuration page for the SonarQube Scanner. The breadcrumb navigation at the top is 'Jenkins / Manage Jenkins / Tools'. Below the header, there is a button '+ Add SonarScanner for MSBuild'. The main section is titled 'SonarQube Scanner installations' and contains a button '+ Add SonarQube Scanner'. A configuration card for 'SonarQube Scanner' is shown with a red close button in the top right corner. Inside the card, the 'Name' field is set to 'Sonar'. The 'Install automatically' checkbox is checked. Below this, there is a section titled 'Install from Maven Central' with a red close button. The 'Version' dropdown menu is set to 'SonarQube Scanner 8.0.1.6346'. At the bottom of the card is a button '+ Add Installer'. Below the configuration card is another button '+ Add SonarQube Scanner'.

6)jenkins --> tools --> dependency check

The screenshot shows the Jenkins 'Tools' configuration page for the Dependency-Check tool. The breadcrumb navigation at the top is 'Jenkins / Manage Jenkins / Tools'. Below the header, there is a button '+ Add Dependency-Check'. The main section is titled 'Dependency-Check installations' and contains a button '+ Add Dependency-Check'. A configuration card for 'Dependency-Check' is shown with a red close button in the top right corner. Inside the card, the 'Name' field is set to 'dc'. The 'Install automatically' checkbox is checked. Below this, there is a section titled 'Install from github.com' with a red close button. The 'Version' dropdown menu is set to 'dependency-check 12.2.0'. At the bottom of the card is a button '+ Add Installer'. Below the configuration card is another button '+ Add Dependency-Check'.

Email notification configuration

jenkins --> system --> email notification

 Jenkins / Manage Jenkins ▾ / System ▾ Q

E-mail Notification

SMTP server

Default user e-mail suffix ?

Advanced ^  Edited

☒ Use SMTP Authentication ?

User Name

 For security when using authentication it is recommended to enable either TLS or SSL

Password

☒ Use SSL ?
☒ Use TLS

SMTP Port ?

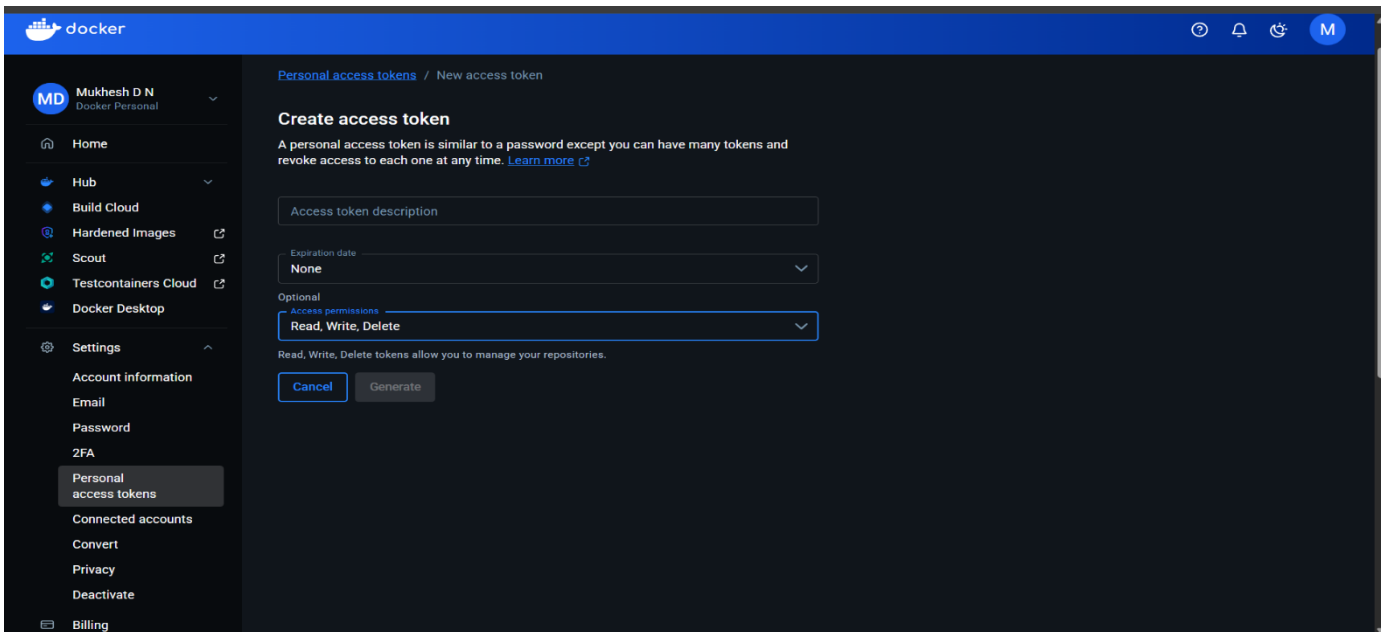
Summary

Save

Apply

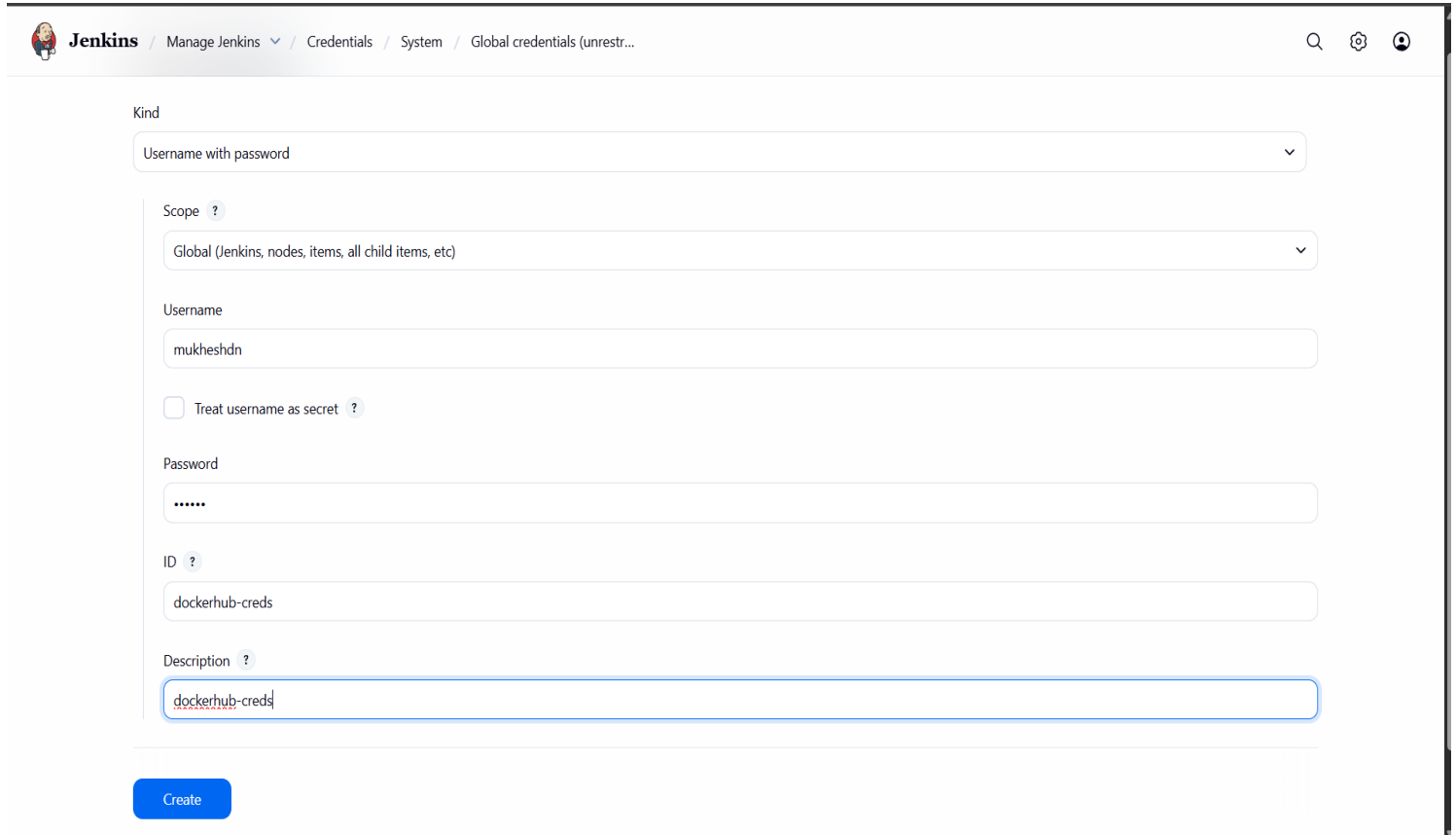
Docker and github credentials setup

1) Create personal access token in docker hub



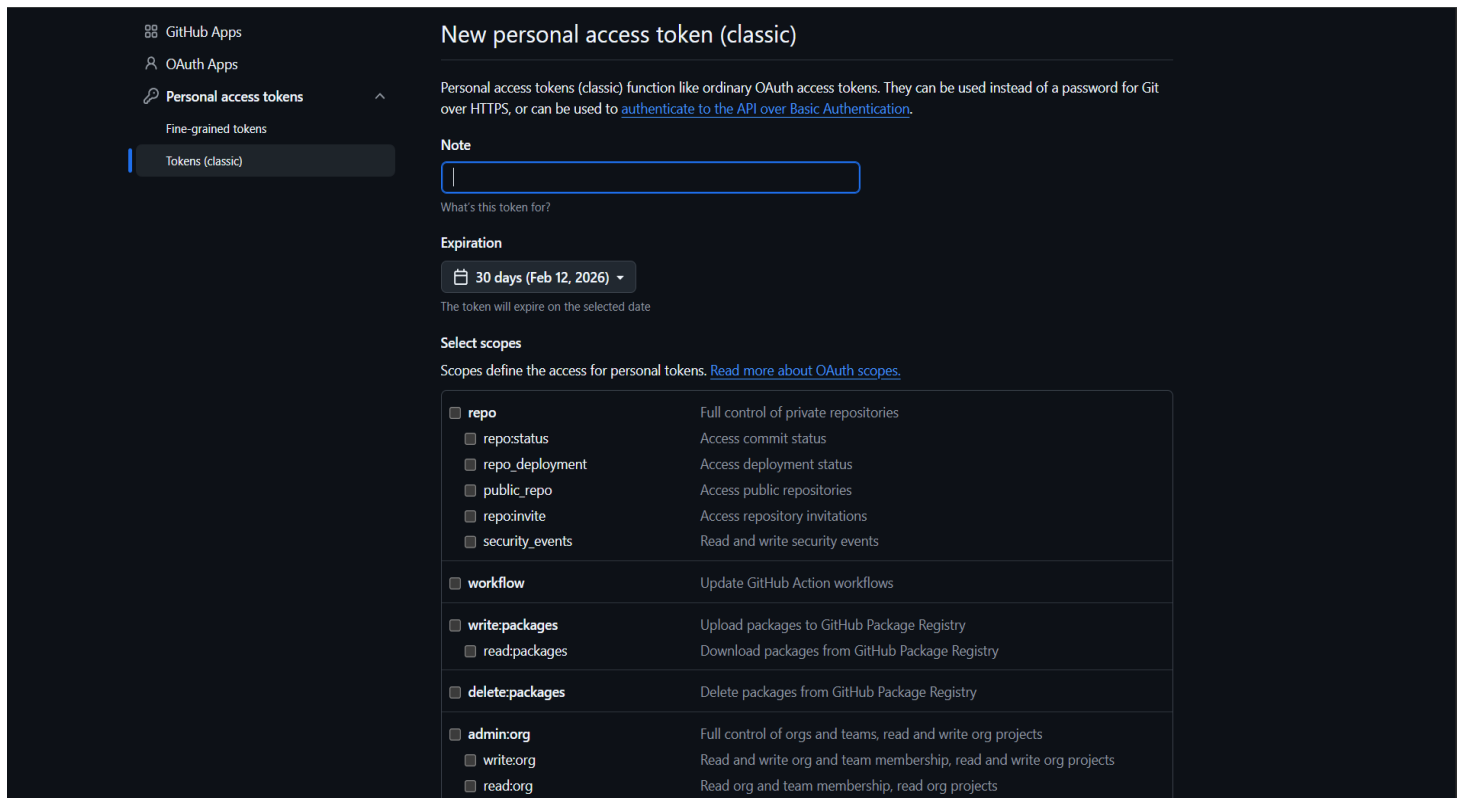
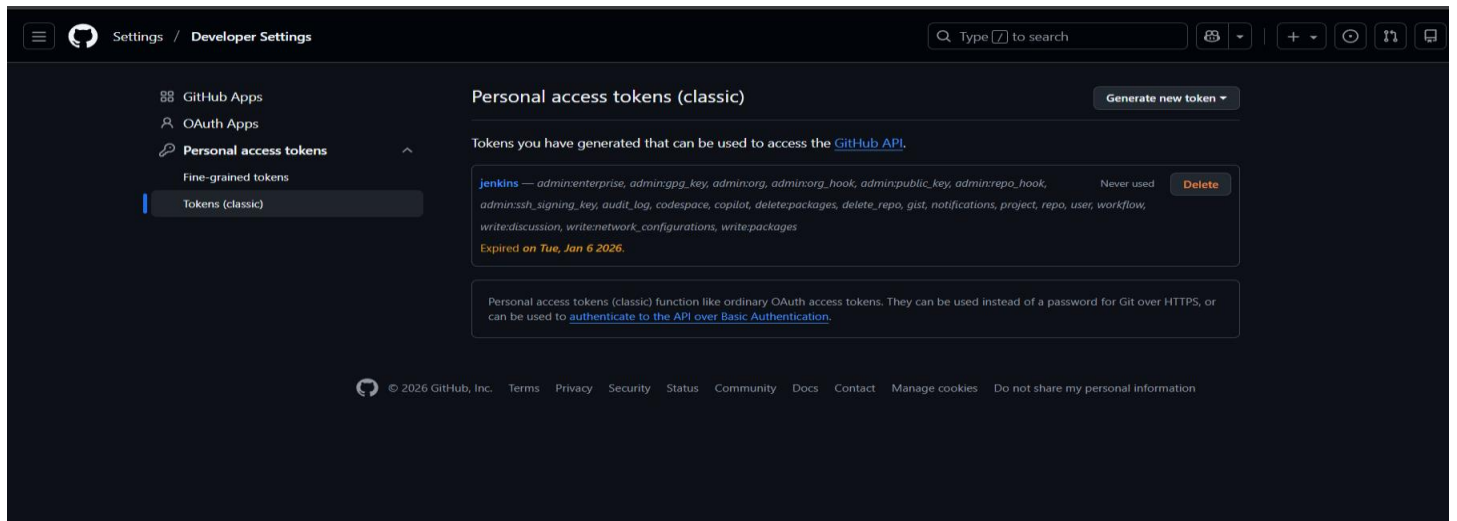
The screenshot shows the Docker Hub interface for a user named 'Mukhesh D N'. The left sidebar contains navigation links: Home, Hub, Build Cloud, Hardened Images, Scout, Testcontainers Cloud, Docker Desktop, Settings, Account information, Email, Password, 2FA, Personal access tokens (highlighted), Connected accounts, Convert, Privacy, Deactivate, and Billing. The main content area is titled 'Personal access tokens / New access token' and 'Create access token'. It includes a description: 'A personal access token is similar to a password except you can have many tokens and revoke access to each one at any time. [Learn more](#)'. Below this are input fields for 'Access token description', 'Expiration date' (set to 'None'), and 'Optional Access permissions' (set to 'Read, Write, Delete'). At the bottom are 'Cancel' and 'Generate' buttons.

2) Add this in Jenkins credentials



The screenshot shows the Jenkins 'Global credentials (unrestricted)' page. The breadcrumb trail is 'Jenkins / Manage Jenkins / Credentials / System / Global credentials (unrestricted)'. The 'Kind' dropdown is set to 'Username with password'. The 'Scope' dropdown is set to 'Global (Jenkins, nodes, items, all child items, etc)'. The 'Username' field contains 'mukheshdn'. There is an unchecked checkbox for 'Treat username as secret'. The 'Password' field is masked with dots. The 'ID' field contains 'dockerhub-creds'. The 'Description' field contains 'dockerhub-creds'. A blue 'Create' button is at the bottom left.

1) Create personal access token (classic) in github



Final credentials in Jenkins

 **Jenkins** / [Manage Jenkins](#) / [Credentials](#) / [System](#) / [Global credentials \(unrestr...](#)

Global credentials (unrestricted)

+ Add Credentials

Credentials that should be available irrespective of domain specification to requirements matching.

 mukheshdn/***** (dockerhub-creds) dockerhub-creds - dockerhub-creds	 ...
 your username/***** (github-token) github-token - github-token	 ...
 Sonar Sonar - Sonar	 ...

```
sudo usermod -aG docker jenkins
```

```
sudo systemctl restart jenkins
```

Jenkins pipeline setup

Select the job pipeline

 Jenkins

jenkins

Configuration

Q

⚙

👤

Configure

General

Triggers

Pipeline

Advanced

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/MukheshDN4352/SocialEcho-1.git

Credentials ?

- none -

+ Add

Advanced

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/master

+ Add Branch

jenkins file link

<https://github.com/MukheshDN4352/DevSecOps-Project/blob/main/Jenkinsfile>

Kubernetes setup

PreRequisites

Before setting up and configuring Kubernetes on AWS using Amazon EKS, ensure that the following prerequisites are in place:

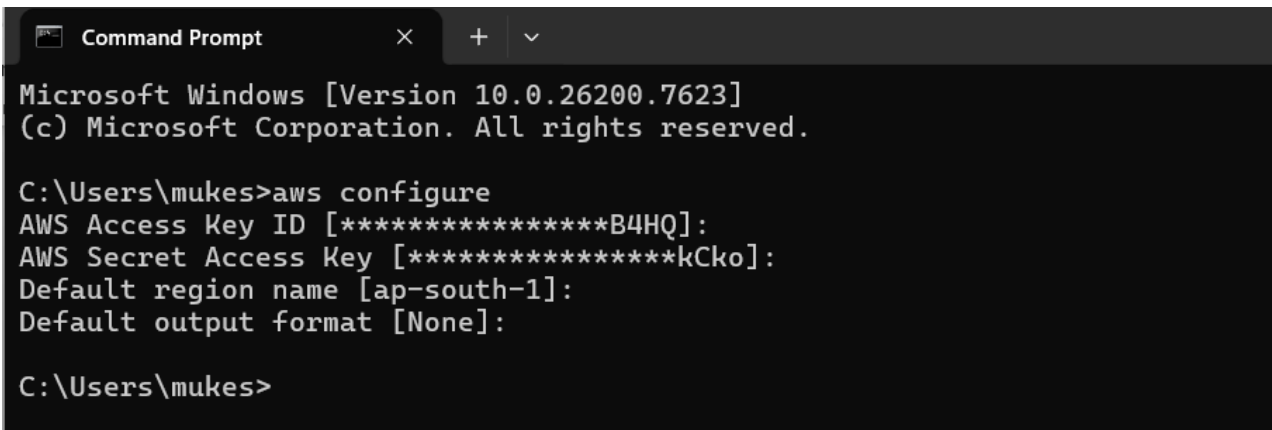
- **AWS Account**
An active AWS account to provision and manage the EKS cluster and related cloud resources.
- **AWS CLI**
The AWS Command Line Interface installed and configured with valid credentials and a default region.
- **EKS CLI (eksctl)**
The eksctl tool installed to create and manage the Amazon EKS cluster.
- **kubectrl**
The Kubernetes command-line tool (kubectrl) installed to interact with the EKS cluster.
- **Helm**
The **Helm package manager** installed to deploy and manage Kubernetes applications and third-party tools (e.g., ArgoCD, Nginx Ingress, monitoring tools).

Kubernetes Setup Overview

- **Node Groups** were created for the EKS cluster to manage worker nodes and support scalable application workloads.
- **Persistent Volumes** were configured to provide durable storage for stateful components.
- The **AWS EBS CSI Driver** was installed and used to dynamically provision persistent volumes.
- **Ingress** was configured to manage external access to services running inside the cluster.
- An **AWS ALB Ingress Controller / AWS Load Balancer Controller** was deployed to provision and manage Application Load Balancers for Kubernetes services.
- **ArgoCD** was deployed on the cluster using **Helm** to enable GitOps-based continuous deployment.
- **Prometheus** and **Grafana** were deployed within the cluster for monitoring and observability.
- All required **commands and configuration steps** for the above components are provided in the sections below.

Cluster setup

- 1) Configure to your aws account



```
Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\mukes>aws configure
AWS Access Key ID [*****B4HQ]:
AWS Secret Access Key [*****kCko]:
Default region name [ap-south-1]:
Default output format [None]:

C:\Users\mukes>
```

- 2) `eksctl create cluster --name test --region ap-south-1 --without-nodes`

Creating node groups

1. `eksctl create nodegroup --cluster test --region ap-south-1 --name backend-ng --node-type c5.large --nodes 2 --nodes-min 2 --nodes-max 6 --node-volume-size 40 --node-volume-type gp3 --tags "project=socialecho,env=prod,component=backend" --node-labels "app=backend,tier=backend"`
2. `eksctl create nodegroup --cluster test --region ap-south-1 --name db-ng --node-type r5.large --nodes 1 --nodes-min 1 --nodes-max 1 --node-volume-size 100 --node-volume-type gp3 --tags "project=socialecho,env=prod,component=database" --node-labels "app=database,tier=storage"`
3. `eksctl create nodegroup --cluster test --region ap-south-1 --name backend-canary-ng --node-type c5.large --nodes 1 --nodes-min 1 --nodes-max 3 --node-volume-size 20 --node-volume-type gp3 --tags "project=socialecho,env=canary,component=backend-canary" --node-labels "app=backend,version=canary,tier=backend-canary"`
4. `eksctl create nodegroup --cluster test --region ap-south-1 --name frontend-canary-ng --node-type t3.medium --nodes 2 --nodes-min 1 --nodes-max 5 --node-volume-size 30 --node-volume-type gp3 --tags "project=socialecho,env=canary,component=frontend-canary" --node-labels "app=frontend,version=canary,tier=frontend-canary"`

5. `eksctl create nodegroup --cluster test --region ap-south-1 --name frontend-ng --node-type t3.medium --nodes 2 --nodes-min 2 --nodes-max 4 --node-volume-size 30 --node-volume-type gp3 --tags "project=socialecho,env=prod,component=frontend" --node-labels "app=frontend,tier=frontend"`

associate oidc provider

```
eksctl utils associate-iam-oidc-provider --region=ap-south-1 --cluster=test --approve
```

installing CSI driver

```
eksctl create iamserviceaccount --name ebs-csi-controller-sa --namespace kube-system --cluster test --role-name AmazonEKS_EBS_CSI_DriverRole --role-only --attach-policy-arn arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy --approve
```

```
eksctl create addon --name aws-ebs-csi-driver --cluster test --service-account-role-arn arn:aws:iam::<AWS_ACCOUNT_ID>:role/AmazonEKS_EBS_CSI_DriverRole --force
```

Apply all the deployments.yaml files in the order given below

```
kubectl apply -f namespace.yaml
```

```
kubectl apply -f secrets.yaml
```

```
kubectl apply -f mongo-server.yaml
```

```
kubectl apply -f mongo-express-server.yaml
```

```
kubectl apply -f node-canary.yaml
```

```
kubectl apply -f node-server.yaml
```

```
kubectl apply -f frontend-server.yaml
```

```
kubectl apply -f frontend-canary.yaml
```

Installing ALB controller

```
curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.11.0/docs/install/iam_policy.json
```

```
aws iam create-policy --policy-name AWSLoadBalancerControllerIAMPolicy --policy-document file://iam_policy.json
```

```
eksctl create iamserviceaccount --cluster=test --namespace=kube-system --name=aws-load-balancer-controller --role-name AmazonEKSLoadBalancerControllerRole --attach-policy-arn=arn:aws:iam:: <AWS_ACCOUNT_ID>::policy/AWSLoadBalancerControllerIAMPolicy --approve
```

```
helm repo add eks https://aws.github.io/eks-charts
```

```
helm repo update eks
```

```
helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system --set clusterName=test --set serviceAccount.create=false --set serviceAccount.name=aws-load-balancer-controller --set region=ap-south-1 --set vpcId=<VPC_ID_OF_THE_CLUSTER>
```

verification

```
kubectl get deployment -n kube-system aws-load-balancer-controller
```

```
kubectl apply -f ingress.yaml
```

Access the application using the DNS name generated by the AWS Load Balancer.

ArgoCD setup

Prerequisites

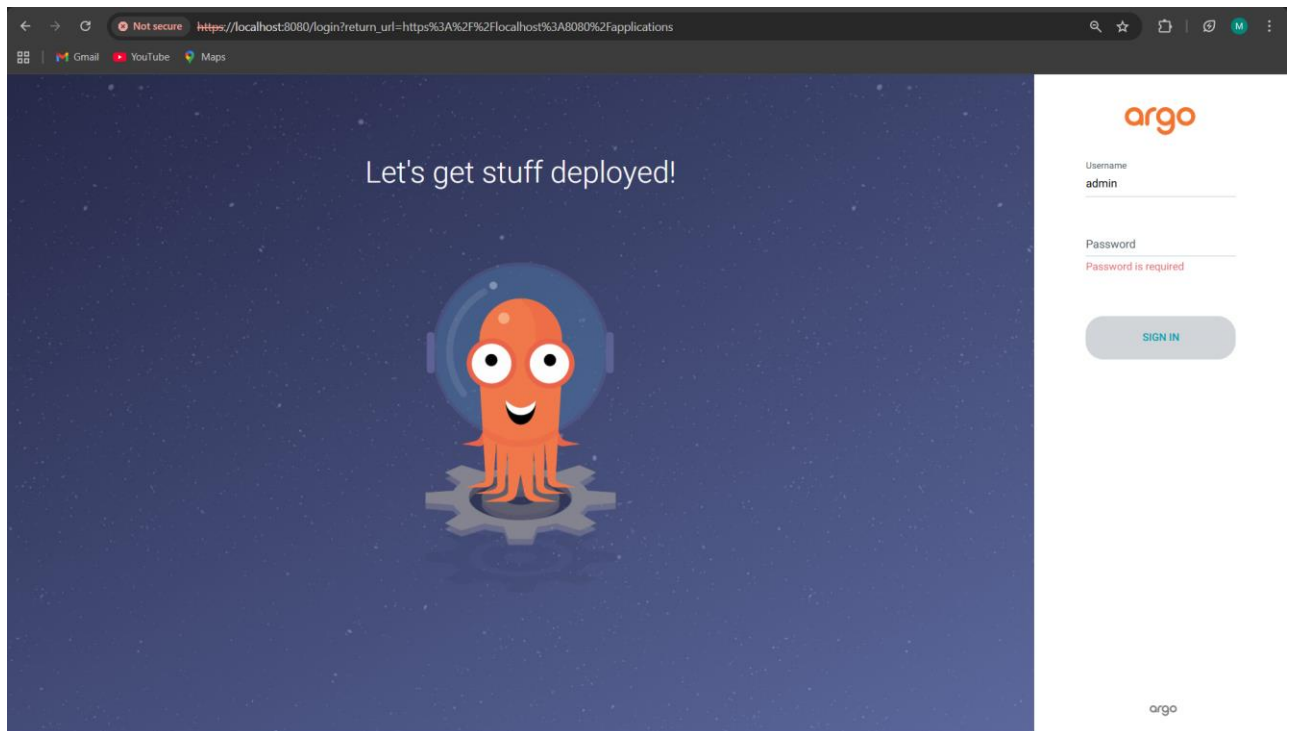
Before deploying and configuring ArgoCD in the Kubernetes cluster, ensure that the following prerequisites are in place:

- **Active Kubernetes Cluster**
A running and accessible Kubernetes cluster (Amazon EKS) with worker nodes in a Ready state.
- **kubectl Configured**
The kubectl CLI installed and configured with the correct kubeconfig to access the target cluster.

Commands

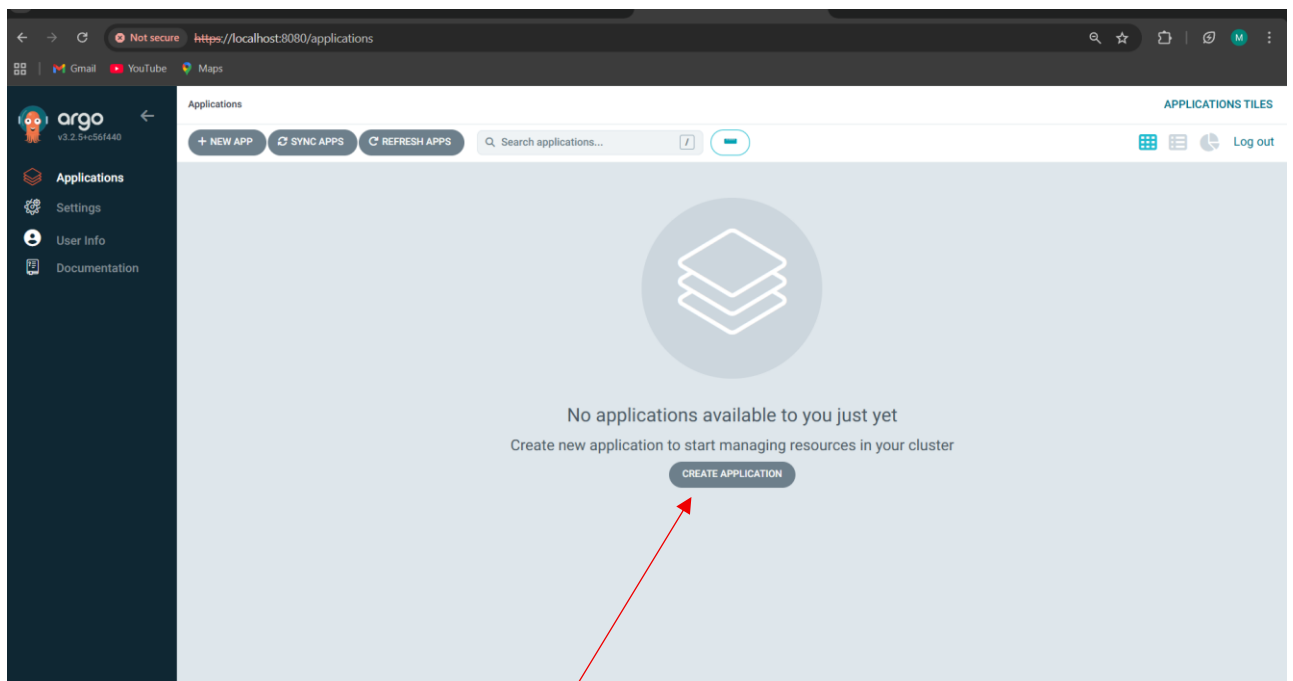
- **Kubectl create namespace argocd**
- **kubectl apply -n argocd -f <https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml>**
- **kubectl get pods -n argocd**
- **kubectl port-forward svc/argocd-server -n argocd 8080:443**

Access the application in browser via <http://localhost:8080>



To get the password

```
kubectl get secret argocd-initial-admin-secret -n argocd -o jsonpath="{.data.password}" | base64 --decode
```



Click **create application**

CREATE

CANCEL

×

GENERAL

EDIT AS YAML

Application Name

argoCD

Project Name

default

SYNC POLICY

Automatic

▼

☒ ENABLE AUTO-SYNC

☐ PRUNE RESOURCES

CREATE

CANCEL

×

SOURCE

Repository URL

https://github.com/MukheshDN4352/eks-gitops-k8s-manifests.git

GIT ▼

Revision

HEAD

Branches ▼

●

Path

kubernetes

DESTINATION

Cluster URL

https://kubernetes.default.svc

URL ▼

Namespace

argocd

Give you kubernetes git hub repo manifest and give the correct path where your yaml files are present

Click on Create

Not secure https://localhost:8080/applications/argocd/argocd?view=tree&resource=

argo v3.2.5+c56f440

Applications Settings User Info Documentation

Resource filters

NAME

KINDS

SYNC STATUS

HEALTH STATUS

APPLICATION DETAILS TREE

DETAILS DIFF SYNC SYNC STATUS HISTORY AND ROLLBACK DELETE REFRESH

APP HEALTH **Progressing**

SYNC STATUS **Synced to HEAD (58443da)**

LAST SYNC **Sync OK to 58443da**

Auto sync is enabled.
Author: Mukhesh DN <154916842+MukheshDN4352@users...
Comment: Update node-server.yaml

Succeeded 2 minutes ago (Wed Jan 21 2026 23:00:52 GMT+0530)
Author: Mukhesh DN <154916842+MukheshDN4352@users...
Comment: Update node-server.yaml

backend-secret
secret
2 minutes

backend
deploy
2 minutes rev1

backend-796dd4bd46
rs
Progressing 9 pods
2 minutes rev1

backend-canary
deploy
2 minutes rev1

backend-canary-d04db47f9
rs
Progressing 1 pods
2 minutes rev1

argocd
2 minutes

frontend-canary
deploy
2 minutes rev1

frontend-canary-7fc79bf69
rs
Progressing 1 pods
2 minutes rev1

frontend-stable
deploy
2 minutes rev1

frontend-stable-565f976fb9
rs
Progressing 9 pods
2 minutes rev1

Prometheus and Grafana

Prerequisites

Before deploying and configuring Prometheus and Grafana in the Kubernetes cluster, ensure that the following prerequisites are in place:

- **Active Kubernetes Cluster (EKS)**
A running and accessible Amazon EKS cluster with all worker nodes in a Ready state.
- **kubectl Configured**
The kubectl CLI installed and configured with the correct kubeconfig to communicate with the EKS cluster.
- **Helm Installed**
The Helm package manager installed to deploy Prometheus and Grafana using Helm charts.

Commands

1. helm repo add stable <https://charts.helm.sh/stable>
2. helm repo add prometheus-community <https://prometheus-community.github.io/helm-charts>
3. kubectl create namespace prometheus
4. helm install stable prometheus-community/kube-prometheus-stack -n prometheus
5. kubectl get pods -n prometheus

```
C:\Users\mukes>kubectl get pods -n prometheus
NAME                                READY   STATUS    RESTARTS   AGE
alertmanager-stable-kube-prometheus-sta-alertmanager-0  2/2     Running   0           106s
prometheus-stable-kube-prometheus-sta-prometheus-0      2/2     Running   0           105s
stable-grafana-646cfcfc5f-q597k                         3/3     Running   0            2m8s
stable-kube-prometheus-sta-operator-667c9d44d4-4fnjb     1/1     Running   0            2m8s
stable-kube-state-metrics-599d5b459c-nvtx5               1/1     Running   0            2m8s
stable-prometheus-node-exporter-lqq4z                   1/1     Running   0            2m8s
```

- 6.kubectl get svc -n prometheus

```
C:\Users\mukes>kubectl get svc -n prometheus
NAME                                TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)                                AGE
alertmanager-operated              ClusterIP    None             <none>            9093/TCP,9094/TCP,9094/UDP            2m47s
prometheus-operated                 ClusterIP    None             <none>            9090/TCP                              2m46s
stable-grafana                      ClusterIP    10.105.113.154   <none>            80/TCP                                3m9s
stable-kube-prometheus-sta-alertmanager  ClusterIP    10.109.153.172   <none>            9093/TCP,8080/TCP                    3m9s
stable-kube-prometheus-sta-operator     ClusterIP    10.101.133.143   <none>            443/TCP                                3m9s
stable-kube-prometheus-sta-prometheus   ClusterIP    10.100.16.254    <none>            9090/TCP,8080/TCP                    3m9s
stable-kube-state-metrics             ClusterIP    10.102.8.61      <none>            8080/TCP                              3m9s
stable-prometheus-node-exporter        ClusterIP    10.102.30.185    <none>            9100/TCP                              3m9s
```

Expose prometheus and grafana to access

kubectrl edit svc stable-kube-prometheus-sta-prometheus -n prometheus

```
File Edit View
self-monitor: "true"
name: stable-kube-prometheus-sta-prometheus
namespace: prometheus
resourceVersion: "696"
uid: cb939787-e6a6-4294-be18-e3e55f666389
spec:
  clusterIP: 10.100.16.254
  clusterIPs:
  - 10.100.16.254
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - name: http-web
    port: 9090
    protocol: TCP
    targetPort: 9090
  - appProtocol: http
    name: reloader-web
    port: 8080
    protocol: TCP
    targetPort: reloader-web
  selector:
    app.kubernetes.io/name: prometheus
    operator.prometheus.io/name: stable-kube-prometheus-sta-prometheus
  sessionAffinity: None
  type: ClusterIP
status:
  loadBalancer: {}

Ln 1, Col 1 | 1,385 characters | Plain text | 100% | Windows (CRLF) | UTF-8
```

```
File Edit View
uid: cb939787-e6a6-4294-be18-e3e55f666389
spec:
  allocateLoadBalancerNodePorts: true
  clusterIP: 10.100.16.254
  clusterIPs:
  - 10.100.16.254
  externalTrafficPolicy: Cluster
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - name: http-web
    nodePort: 30617
    port: 9090
    protocol: TCP
    targetPort: 9090
  - appProtocol: http
    name: reloader-web
    nodePort: 32194
    port: 8080
    protocol: TCP
    targetPort: reloader-web
  selector:
    app.kubernetes.io/name: prometheus
    operator.prometheus.io/name: stable-kube-prometheus-sta-prometheus
  sessionAffinity: None
  type: LoadBalancer
status:
  loadBalancer: {}

Ln 1, Col 1 | 1,501 characters | Plain text | 100% | Windows (CRLF) | UTF-8
```

kubectl edit svc stable-grafana -n prometheus

```
File Edit View
app.kubernetes.io/managed-by: helm
app.kubernetes.io/name: grafana
app.kubernetes.io/version: 12.3.1
helm.sh/chart: grafana-10.5.8
name: stable-grafana
namespace: prometheus
resourceVersion: "1936"
uid: 8eae98de-5ce8-4fa5-8dee-d1ea94764076
spec:
  allocateLoadBalancerNodePorts: true
  clusterIP: 10.105.113.154
  clusterIPs:
  - 10.105.113.154
  externalTrafficPolicy: Cluster
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - name: http-web
    nodePort: 31210
    port: 80
    protocol: TCP
    targetPort: grafana
  selector:
    app.kubernetes.io/instance: stable
    app.kubernetes.io/name: grafana
  sessionAffinity: None
  type: ClusterIP
status:
  loadBalancer: {}

Ln 42, Col 18 | 1,187 characters | Plain text | 100% | Windows (CRLF) | UTF-8
```

```
File Edit View
app.kubernetes.io/managed-by: helm
app.kubernetes.io/name: grafana
app.kubernetes.io/version: 12.3.1
helm.sh/chart: grafana-10.5.8
name: stable-grafana
namespace: prometheus
resourceVersion: "1936"
uid: 8eae98de-5ce8-4fa5-8dee-d1ea94764076
spec:
  allocateLoadBalancerNodePorts: true
  clusterIP: 10.105.113.154
  clusterIPs:
  - 10.105.113.154
  externalTrafficPolicy: Cluster
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - name: http-web
    nodePort: 31210
    port: 80
    protocol: TCP
    targetPort: grafana
  selector:
    app.kubernetes.io/instance: stable
    app.kubernetes.io/name: grafana
  sessionAffinity: None
  type: LoadBalancer
status:
  loadBalancer: {}

Ln 1, Col 1 | 1,190 characters | Plain text | 100% | Windows (CRLF) | UTF-8
```

Access the both the dashboards with **external Ip** created by **LoadBalancer**

The complete cluster data can be visualized in Grafana dashboards, including all metrics related to pod health and performance.

**Thank
you**